

▪ ClearTax OMAN E-INVOICING

OMAN OFFLINE B2C E-INVOICE GENERATION

B2C e-Invoicing SDK

This SDK is a Java library that **validates invoices** locally at the point of sale and generates **Oman-compliant B2C QR code, the Invoice XML and Seller UUID** - **fully offline**, after a one-time online device onboarding.

Sellers and System Integrator (SI) partners integrate the SDK into a Point-of-Sale (POS) system.

AUDIENCE	Sellers and System Integrator (SI) partners integrate the SDK into a Point-of-Sale (POS) system.
PRODUCT	com.cleartax.oman:oman-einvoicing-sdk
VERSION	1.4.0
REQUIRES	Java 17 or above

Table of contents

- 01. What the SDK does (and does not do)
- 02. Key concepts
- 03. Installation
- 04. Onboarding the device (one-time, online)
- 05. Generate XML and QR code
- 06. The Java API
- 07. CLI reference
- 08. Configuration
- 09. Error handling
- 10. Storage, logging & background tasks
- 11. Updates & revocation
- 12. Getting support
- 13. Troubleshooting / FAQ

1 What the SDK does (and does not do)

It does:

- **Validate** invoices locally (UBL 2.1 XSD → PINT Schematron → OTA Oman rules).
- Generate the **Seller UUID** (BTOM-004, UUID v5) offline for each invoice.
- Encode and generate the **B2C QR code** (TLV → Base64, ≤ 700 characters) offline for each invoice.
- Render a **printable QR image** (PNG) for the receipt.
- Create, store and sign the **reporting XML** of each invoice on your system.

It does *not*:

- Generate the **Invoice UUID** (BTOM-002) - that is created by the ClearTax backend during invoice reporting.
- **Sign the QR** or perform any OTA-facing cryptography - the Oman B2C QR is **unsigned**.
- **Submit to OTA / Peppol** - the ClearTax backend builds the Tax Data Document (TDD) and submits it via AS4. Getting the stored reporting copies to that backend is **your** integration's job (Section 10).
- **Auto-update itself** - it only notifies you when a newer version is available.

The onboarding, integrity, and invoice-signing mechanisms in this SDK are **ClearTax-internal** (licensing, tamper detection, source authenticity). They have **no effect** on the QR contents, OTA submission, or the Peppol network.

2 Key concepts

Term	Meaning
OTA	Oman Tax Authority - the Omani e-invoicing regulator.
B2C	Business-to-consumer simplified invoice - the scope of this SDK.
Seller UUID (BTOM-004)	A per-invoice UUID v5 the SDK generates and embeds in the QR + PINT XML.
TLV	Tag-Length-Value encoding scheme used for the QR payload.
QR code Invoice type	<code>STANDARD_B2C</code> (Type 01) or <code>PROFIT_MARGIN</code> (Type 02).
Device onboarding	A one-time online step that licenses one SDK install to one machine.
Device fingerprint	A stable hardware/OS identifier that binds the license to one machine.
.sig file	A companion signature file written next to each invoice XML, verified by the backend.

3 Installation

The SDK is **distributed as a JAR**, not published to Maven Central. Your ClearTax representative provides sdk.jar during onboarding. One artifact serves both roles: it is the library you compile against and the JAR the sdk CLI launcher runs (see Section 7).

With Maven or Gradle. Install the JAR into a repository your build can resolve, then depend on it as normal. For a single developer, the local repository is enough:

```
mvn install:install-file -Dfile=sdk.jar \
-DgroupId=com.cleartax.oman -DartifactId=oman-einvoicing-sdk \
-Dversion=1.4.0 -Dpackaging=jar
```

For a team, publish it once to your own repository manager (Nexus, Artifactory) with `mvn deploy:deploy-file` instead, so every developer and your CI resolve the same artifact. Either way the coordinate is then ordinary:

```
<dependency>
  <groupId>com.cleartax.oman</groupId>
  <artifactId>oman-einvoicing-sdk</artifactId>
  <version>1.4.0</version>
</dependency>

// build.gradle
implementation 'com.cleartax.oman:oman-einvoicing-sdk:1.4.0'
```

Without a build tool. Put the JAR on the compile and runtime classpath directly (`java -cp sdk.jar:...`).

- **Java 17 or newer** is required.
- The JAR is **shaded**: it carries its own dependencies and adds no transitive downloads to your build.
- Do **not** repackage, shade, or modify the JAR: the SDK verifies its own integrity at startup, and a modified binary will refuse to run (see Section 11).

4 Onboard the device (one-time, online)

Each POS machine must be activated **once**, while online. After that it operates offline indefinitely.

1. A seller admin creates a POS slot for the VAT number in the ClearTax portal and receives a **one-time activation code**.
2. On the POS machine, run:

```
java -jar sdk.jar onboard --vat OM1234567891 --code A7K2-NX9P-QM4R-TZ8W  
--auth-token <seller-token>
```

The activation code is **16 characters in 4x4 groups**, valid **48 hours**. `--auth-token` is your ClearTax seller auth token (sent as the `x-clear-tax-auth-token` header).

3. On success:

```
Device successfully onboarded. Device ID: f1e2d3c4-...  
SDK is ready for offline invoice operations.
```

Behind the scenes the SDK generates a device keypair, computes a device fingerprint, calls the activation API (sending its **public key**), and stores the returned licence token plus the device **private key encrypted on disk**.

BULK ONBOARDING (100+ STORES)

SI partners do **not** loop this CLI per machine. Use the device activation API directly (batch path). Both paths register the device in the same dashboard. Contact ClearTax for API access.

You can also onboard programmatically from Java - see `onboard(...)`.

5 Generate QR codes (offline, per invoice)

After onboarding, all invoice operations require **zero network**.

```
import com.cleartax.oman.sdk.OmanSdk;
import com.cleartax.oman.sdk.model.QrResult;
import com.cleartax.oman.sdk.error.OmanErrorCode;

// Initialize ONCE at process start - reuse for every invoice (thread-safe, long-lived).
OmanSdk sdk = OmanSdk.initialize();

// JSON -> UBL -> validate -> Seller UUID -> TLV QR -> store + sign
QrResult result = sdk.generateFromJson(invoiceJson);

if (result.isSuccess()) {
    String base64Tlv = result.getQrBase64(); // ≤ 700 chars
    byte[] qrImagePng = result.getQrImagePng(); // print this on the receipt
    String sellerUuid = result.getSellerUuid(); // BTOM-004
} else {
    OmanErrorCode code = result.getErrorCode(); // typed - never a bare exception
    // show result.getErrorMessage() / result.getViolations() to the operator
}
```

INITIALIZE ONCE.

OmanSdk is thread-safe and designed to be a long-lived singleton in the POS process. The startup integrity checks run at `initialize()` - not per invoice - so checkout latency is unaffected.

6 The Java API

The **only** public entry point is `com.cleartax.oman.sdk.OmanSdk`. All integrations use this facade plus a small set of immutable value/error types (`model/*`, `error/*`). Everything under `...sdk.internal.*` is private and may change without notice.

6.1 Lifecycle

```
OmanSdk sdk = OmanSdk.initialize();           // default config (env / system properties)
OmanSdk sdk = OmanSdk.initialize(SdkConfig config); // explicit config

SdkStatus status = sdk.status(); // INITIALIZING | READY | HALTED(reason) | REVOKED
```

What each status implies:

1. INITIALIZING → still starting up
2. READY → good to go
3. HALTED → something's broken (e.g. failed a security/integrity check), the QR and validation entry points will refuse to run until the cause is fixed.
4. REVOKED → this device's access has been shut off

6.2 Core methods

Method	Purpose
<code>QrResult generateFromJson(String json)</code>	The full path. Invoice JSON → PINT OM UBL → validate (Stages 1–3) → Seller UUID → TLV QR. On success, and only once the device is onboarded, also persists the invoice XML + <code>.sig</code> for deferred reporting.
<code>QrResult generateQrFromFields(InvoiceFields fields)</code>	For POS systems that have fields in memory but no UBL XML. Skips XSD/Schematron; still applies field rules, UUID, and QR encoding.
<code>ValidationResult validate(String invoiceXml)</code>	Validation only - no QR, no storage/signing side effects. Runs Stages 1–3 and never throws for invalid input.
<code>ValidationResult validateJson(String json)</code>	JSON → UBL → validate, no side effects. Malformed JSON returns a typed <code>MAP_INPUT_INVALID</code> failure rather than throwing.

Method	Purpose
String jsonToUbl(String json)	Maps the SDK's invoice JSON to PINT OM UBL XML. Does not validate - pair with <code>validateJson</code> .
QrResult generate(String invoiceXml)	For callers who already hold PINT OM UBL XML. Validates (Stages 1–3), reads the document back into the invoice model, then runs the same derivation as the JSON path → Seller UUID → TLV QR. Persists + signs the reporting copy once onboarded. Accepts an <code>Invoice</code> or a <code>CreditNote</code> root.

What `generateFromJson()` does, in order: map JSON → UBL → validate (Stage 1 well-formedness → EN16931 PINT Schematron → OTA Oman rules) → generate Seller UUID → encode TLV QR → check length (≤ 700) → generate XML + write `.sig` (once onboarded) → return `QrResult`.

STORAGE/SIGNING NEVER BLOCKS THE RECEIPT.

If persisting or signing the reporting copy fails, the QR is still returned successfully and the issue is surfaced as a **warning** (`result.getWarnings()`) and retried later. The buyer always gets a valid receipt.

6.3 Onboarding & maintenance

Mostly used by the CLI, but available programmatically (all require connectivity):

```
OnboardingResult onboard(String vat, String activationCode, String authToken); //
one-time activation
RevocationStatus checkRevocation(); // writes a local flag if revoked
VersionInfo checkForUpdates(); // version information for the install
PushResult pushPendingArtifacts(); // reports the local queue depth
```

`onboard(...)` activates the device against ClearTax (`POST /v1/pos-devices/activate`) on the active flavor's environment, verifies the returned activation JWT with that flavor's key, and seals the token, the device private key, the seller auth token, and the flavor locally. The activation code is one-shot: if the connection drops after the server consumes it, the next attempt returns `ONB_CODE_USED` - mint a new code rather than retrying.

`checkRevocation()` calls `GET /v1/pos-devices/{deviceId}/status`. Only an explicit `REVOKED` acts (it writes `revoked.flag`, halting the **next** startup); any error - offline, 5xx, unknown

device, rejected token - returns `UNKNOWN` and changes nothing, so a flaky network can never take a till offline.

6.4 Public value types

`QrResult`

```
boolean    isSuccess();
String      getQrBase64();    // TLV Base64, ≤ 700 chars (null on failure)
byte[]      getQrImagePng();  // printable QR image (null on failure)
String      getSellerUuid();  // BTOM-004 (null on failure)
InvoiceType getInvoiceType(); // STANDARD_B2C | PROFIT_MARGIN
OmanErrorCode getErrorCode(); // null on success
String      getErrorMessage(); // human-readable, actionable (null on success)
List<RuleViolation> getViolations(); // populated on validation failure
List<Warning> getWarnings();    // e.g. deferred storage/signing issues
```

`GETINVOICETYPE()` NEVER TELLS YOU IT WAS A CREDIT NOTE.

`InvoiceType` has exactly two constants, and a credit note takes one of them like any other document - `STANDARD_B2C` (01) or `PROFIT_MARGIN` (02), never 381. `QrResult` carries no document-kind assessor at all, because the QR payload itself does not distinguish a refund from a sale (Section 5). Track that on your side from the document you submitted; do not try to recover it from the result.

`InvoiceFields` (structured input for `generateQrFromFields`) - immutable, built with a builder. Accessors have **no get prefix**: `qrVersion()`, `vat()`, `timestamp()`, ...

```
InvoiceFields fields = InvoiceFields.builder() // qrVersion defaults to "1.1" (SRA v1.0.2)
    .invoiceType(InvoiceType.STANDARD_B2C) // required (both) - 01 |
    PROFIT_MARGIN (02)
    .invoiceNumber("INVOICE-01")           // required (both) - IBT-001, new in SRA
    v1.0.2
    .sellerName("SELLERA")                 // required (both) - Arabic-safe (UTF-8)
    .vat("OM1234567891")                   // required (both)
    .date(LocalDate.of(2026, 5, 13))       // required (both) - yyyy-MM-dd
    .timestamp(LocalTime.of(18, 31, 0))     // required (both) - HH:mm:ss
    .totalInclVat(new BigDecimal("302.340")) // required (Type 01)
    .vatTotal(new BigDecimal("14.520"))    // required (Type 01)
    .build();                             // Type 02 uses .totalAmountDue(...) instead
```

ALL FIELDS MARKED **REQUIRED** MUST BE PRESENT AND NON-BLANK

for the invoice type. A `null` or blank required field (e.g. `vat` or `timestamp`) returns `QR_FIELD_MISSING` (Section 9) - the SDK never throws a `NullPointerException` for missing invoice data and never emits a QR with an empty tag.

7 CLI reference

The SDK ships a `sdk` launcher for two kinds of work: **setup/support** (onboarding, log management, status) and **per-invoice tooling** (`validate`, `generate-xml`, `qr`) for testing, support, and batch/one-off use. The per-transaction checkout path is the embedded Java API (Section 5) - the CLI is **not** for the checkout hot path.

Note - Add '`java -jar`' before every command.

Command	Purpose	Key options
<code>sdk.jar onboard</code>	One-time device activation	<code>--vat <VATIN></code> (required), <code>--code <CODE></code> (required), <code>--auth-token <TOKEN></code> (required), <code>--config <path></code>
<code>sdk.jar logs export</code>	Bundle local logs into a support zip	<code>--output <path.zip></code> (required), <code>--since <ISO-timestamp></code>
<code>sdk.jar logs clean</code>	Clear local logs	<code>--force</code> (required to delete logs not yet pushed)
<code>sdk.jar version</code>	Print installed version; if online, report whether a newer one exists	-
<code>sdk.jar status</code>	Run the three startup integrity checks → <code>READY</code> / <code>HALTED<reason></code> / <code>REVOKED</code>	-
<code>sdk.jar validate</code>	Validate one invoice; print each violation	<code>--in <invoice.(xml json)></code> (required), <code>--xml</code> / <code>--json</code>
<code>sdk.jar generate-xml</code>	Map the SDK's clean JSON → PINT OM UBL XML	<code>--in <invoice.json></code> (required), <code>--out <invoice.xml></code> (required)

Command	Purpose	Key options
sdk.jar qr	JSON → UBL → validate → QR (Base64 + PNG)	--in <invoice.json> (required), --out <qr.png>, --print
sdk.jar qr (adds the invoice XML + sig file in the config <path> folder)	JSON → UBL → validate → QR (Base64) → XML + signature file	--in <invoice.json> --json (required) --config <path>
sdk.jar help [command]	Usage	-

Examples

```
java -jar sdk.jar onboard --vat OM1234567891 --code A7K2-NX9P-QM4R-TZ8W
--auth-token <seller-token>
java -jar sdk.jar logs export --output /tmp/logs.zip --since 2026-06-29T00:00:00Z
java -jar sdk.jar logs clean --force
java -jar sdk.jar status
java -jar sdk.jar validate --in invoice.json --json
java -jar sdk.jar generate-xml --in invoice.json --out invoice.xml
java -jar sdk.jar qr --in invoice.json --out qr.png --print
```

`onboard` is the **only** command that requires connectivity; everything else - including `validate`, `generate-xml`, and `qr` - works fully offline.

Exit codes

Exit	Meaning
0	Success
1	Generic / unexpected error - incl. invoice VAL_* / QR_* failures (sdk validate, sdk qr)
2	Invalid arguments / usage
3	Onboarding failed (bad / expired / used code, VAT mismatch)
4	No connectivity (for commands that require it)

Exit	Meaning
5	Integrity check failed / device halted
6	Device revoked

Exit codes map onto the error-code families in Section 9, so scripts and CI can branch on them.

8 Configuration

Pass an `SdkConfig` to `initialize(...)`, or rely on defaults (overridable via environment variables / system properties):

Built with `SdkConfig.builder()`; `SdkConfig.defaults()` resolves everything from environment variables and system properties. Accessors have no `get` prefix.

Setting	Purpose
<code>invoiceStorageDir()</code>	Where invoice XML + <code>.sig</code> are written.
<code>logDir()</code>	Where JSON logs are written.
<code>tokenStorePath()</code>	Encrypted licence token.
<code>signingCredentialPath()</code>	Encrypted device private key.
<code>authTokenPath()</code>	Sealed seller auth token, persisted at onboarding for background status checks.
<code>flavorMarkerPath()</code>	Records the flavor the device was on boarded against.
<code>revokedFlagPath()</code>	Local <code>revoked.flag</code> ; present means the next startup halts.
<code>flavor()</code>	DEV PROD - selects the ClearTax key + base URL. Default PROD.
<code>flavorSource()</code>	Which input selected the flavor (diagnostics).
<code>clearTaxBaseUrl()</code>	Portal / activation / push endpoints; defaults to the flavor's URL. Override with <code>oman.sdk.baseUrl</code> / <code>OMAN_SDK_BASE_URL</code> .
<code>countryCode()</code>	Jurisdiction code.

Setting	Purpose
<code>httpConnectTimeout()</code> / <code>httpRequestTimeout()</code>	Default 5s connect.
<code>logRetention()</code>	Default 24h.
<code>requireOnboarding()</code>	Default false , so a device with no activation token still runs the offline QR and validation paths. Set it to true to require an onboarded device; an un-onboarded install then halts with ONB_NOT_ONBOARDED .

9 Error handling

Every expected failure returns a typed **OmanErrorCode** inside the result object - never a bare exception. `SdkException` (unchecked) is reserved for **programmer errors** (calling `generate()` before `initialize()`, passing a `null` argument such as a null `InvoiceFields/XML/JSON`, a corrupt bundled resource). A null or blank *field within* a supplied `InvoiceFields` is **not** a programmer error - it returns `QR_FIELD_MISSING`. Always include a **default** branch - new codes are added in minor versions.

```
QrResult r = sdk.generateFromJson(json);
if (!r.isSuccess()) {
    switch (r.getErrorCode()) {
        case VAL_XSD_FAILED, VAL_SCHEMATRON_FAILED, VAL_OTA_FAILED ->
            showViolations(r.getViolations()); // fix the invoice, retry
        case QR_PAYLOAD_TOO_LONG ->
            escalate("Receipt cannot be issued - QR payload too long");
        case LIFE_DEVICE_REVOKED ->
            stop("Device revoked - re-onboard if authorized");
        case ONB_NOT_ONBOARDED ->
            prompt("Run: sdk onboard");
        default ->
            log(r.getErrorMessage());
    }
}
```

Error-code catalog

Onboarding & integrity - halt the SDK (exit codes in parentheses)

Code	When	Operator action
ONB_NOT_ONBOARD (5)	A QR/validation entry point called with no licence token, once <code>requireOnboarding</code> is enabled	Run <code>sdk onboard</code> .
ONB_CODE_INVALID (3)	Activation code not found / wrong VAT	Re-issue a code in the portal.
ONB_CODE_EXPIRED (3)	Activation code past its validity window	Re-issue a code.
ONB_CODE_USED (3)	Activation code already consumed	Re-issue a code.
ONB_VAT_MISMATCH (3)	Code exists but was issued for a different VAT	Use the code under its issued VAT, or re-issue.
ONB_NO_CONNECTIVITY (4)	Onboarding could not reach ClearTax	Connect to a network and retry.
INTEG_TOKEN_MISSING (5)	Token absent / corrupt / undecryptable	Re-onboard the device.
INTEG_TOKEN_INVALID (5)	Licence signature invalid	Re-install the official SDK / re-onboard.
INTEG_FINGERPRINT_MISMATCH (5)	Token belongs to another machine	Re-onboard this machine.
INTEG_JAR_TAMPERED (5)	SDK binary was modified	Re-install the official SDK.
INTEG_FLAVOR_MISMATCH (5)	Token was issued for a different flavor (<code>DEV</code> vs <code>PROD</code>)	Re-onboard against the correct environment.
CFG_FLAVOR_INVALID (2)	Unrecognized flavor value from config / property / env / CLI	Use <code>DEV</code> or <code>PROD</code> .
CFG_FLAVOR_KEY_UNAVAILABLE (5)	The active flavor's ClearTax key is missing or unreadable	Re-install the official build for that environment.

Code	When	Operator action
LIFE_DEVICE_REVOKED (6)	Device revoked in the portal	Re-onboard if authorized.

Validation - block the QR (fix the invoice and retry)

Code	When
VAL_XSD_FAILED	UBL 2.1 structural validation failed (violations carry XPath + rule IDs)
VAL_SCHEMATRON_FAILED	EN16931 PINT business rule failed (e.g. BR-CO-10)
VAL_OTA_FAILED	OTA Oman business rule failed
VAL_MALFORMED_XML	Input is not parseable XML

QR & UUID - block the QR (compliance-critical)

Code	When
QR_PAYLOAD_TOO_LONG	Base64 TLV > 700 chars - cannot issue receipt; escalate
QR_VALUE_TOO_LONG	A single field's UTF-8 value > 255 bytes (usually the seller name)
QR_FIELD_MISSING	A required field is null or blank for the invoice type (e.g. vat , timestamp , vat_total) - a typed error, never a crash or a broken QR
UUID_INPUT_INVALID	The Seller UUID input string could not be built
MAP_INPUT_INVALID	Invoice JSON could not be mapped to UBL, or QR fields could not be derived from the invoice (JSON path)

Storage, signing & connectivity - WARNINGS (never block the receipt; retried in the background)

Code	When
STORE_WRITE_FAILED	Invoice XML could not be persisted
SIGN_FAILED	.sig could not be produced
STORE_DISK_FULL	No disk space for the reporting copy
NET_PUSH_FAILED	Log / XML push could not reach ClearTax (will retry)
NET_OFFLINE	A background online task found no connectivity (deferred)
LIFE_UPDATE_AVAILABLE	Informational: a newer SDK version exists (not an error)

10 Storage, logging & background tasks

- **Invoice storage:** each generated invoice XML and its `.sig` companion are written to `invoiceStorageDir`. The SDK does not transmit them, and nothing removes them - they stay on disk until your integration collects them.
- **Logs:** structured JSON lines with `timestamp`, `level`, `operation`, `device_id`, `sdk_version`, `vat`, `result`, `error_code`, `duration_ms`, written to `logDir`. Files older than `logRetention` (default 24h) are **deleted**, so export anything support may need before it ages out (Section 12).
- **Runs only when online, never blocking checkout:** the revocation check. Every invoice operation itself needs **zero network**.

DRAINING THE REPORTING QUEUE IS THE SI'S RESPONSIBILITY.

The SDK signs each invoice copy, writes it to `invoiceStorageDir`, and stops there - it never uploads anything. Your integration must collect those files and deliver them to the ClearTax backend, which builds the Tax Data Document and submits it to OTA. `pushPendingArtifacts()` reports how many are queued locally; it does **not** send them. Nothing prunes the directory either, so a terminal that is never drained grows without bound while holding the only copy of your reporting evidence. Size and monitor that volume, and never delete a file until the backend has confirmed receipt.

11 Updates & revocation

Version awareness (no auto-update). Whenever the SDK connects to ClearTax it checks for a newer version. If one exists it **logs a notification and surfaces it via CLI output** (version number + download reference). Installing the update is the IT admin's / SI partner's responsibility.

```
VersionInfo v = sdk.checkForUpdates();
if (v.isUpdateAvailable()) {
    // "A newer SDK version {v.latest()} is available. Current: {v.current()}. It will NOT
    auto-update."
}
```

A compliance-critical spec change (UUID format, QR tag, rule set) ships as a **new version**. Upgrade promptly so an outdated install does not silently produce non-compliant invoices.

Integrity checks. On every startup the SDK verifies its licence token and that the token was issued for **this** machine. Any failure halts the SDK with a specific, actionable message (see Section 9).

Revocation. An admin can revoke a device in the portal. The next time that device is online it detects the revocation and, on the following startup, halts with `LIFE_DEVICE_REVOKED`. A permanently-offline device keeps operating until it next connects (accepted trade-off for offline resilience).

12 Getting support

When you hit an issue, bundle the local logs and share them with ClearTax support:

```
sdk logs export --output /tmp/oman-sdk-logs.zip
```

Run `sdk status` first - it reports which, if any, startup integrity check failed and is the fastest first diagnostic. Do **not** run `sdk logs clean` before exporting: by default the SDK refuses to delete logs that have not yet been pushed, but `--force` overrides that and would lose the bundle.

A QR call throws before I call anything. You must call `OmanSdk.initialize()` first, and pass non-null arguments. These are programmer errors and surface as `SdkException` - not `OmanErrorCode`.

Which entry point should I use? `generateFromJson(...)` if you can emit the SDK's invoice JSON - it covers mapping, validation, QR, storage and signing in one call. `generate(xml)` if your system already produces PINT OM UBL. `generateQrFromFields(...)` if you hold only the QR fields. All three share one Seller UUID / TLV derivation, so they cannot disagree.

`generate(xml)` returns `MAP_INPUT_INVALID` for my credit note. Not since **1.4.0** - credit notes now yield a QR on both paths. If you still see this, you are on 1.3.x or earlier, where the UBL path refused credit notes deliberately; upgrade. On 1.4.0+, a credit note that fails does so for the same reasons an invoice would (`VAL_SCHEMATRON_FAILED` if it does not carry `cbc:CreditNoteTypeCode 381`, `QR_FIELD_MISSING` if it lacks an issue time or a monetary total).

QR generation fails with `QR_PAYLOAD_TOO_LONG` / `QR_VALUE_TOO_LONG`. The TLV length is counted in **UTF-8 bytes, not characters** - a long Arabic seller name can exceed the 700-char payload limit or the 255-byte per-value limit. Shorten the offending field (usually the seller name).

The SDK halts at startup. Run `sdk status`. `INTEG_JAR_TAMPERED` means the JAR was modified - re-install the official build. `INTEG_FINGERPRINT_MISMATCH` means the token was issued for a different machine - re-onboard this one.

Do I need the internet for every sale? No. Only **onboarding** requires connectivity. All QR generation is fully offline. The revocation check runs opportunistically when a connection is available; delivering the stored reporting copies to ClearTax is your integration's job (Section 10).

How do I onboard hundreds of stores? Use the device activation API directly (bulk path), not a CLI loop. Contact ClearTax for API access.